Parallel Project Report: Distributed Sparse Matrix-Vector Multiplication with MPI

Student: Ilaria Basanisi (ID: 244528)
Email: ilaria.basanisi@studenti.unitn.it
Course: Introduction to Parallel Computing (2024-2025)
University of Trento

***Abstract*—Sparse Matrix-Vector Multiplication (SpMV) is a fundamental operation in scientific computing, with applications ranging from iterative solvers to graph analytics. This work presents a distributed-memory parallel implementation of SpMV using MPI with 1D cyclic (modulo) partitioning and optimized collective communication via `MPI_Alltoallv`. In this paper we will evaluate strong scaling performance using four matrices from 10K to 2380K and weak scaling using matrices from 10K up to 2560K. Strong scaling results demonstrate good scaling up to 64 processes, achieving 9.59× speedup on the 504K matrix. However, severe scalability degradation occurs beyond 64 processes due to communication overhead dominating computation time. Weak scaling analysis shows near-constant runtime up to 32 processes, followed by gradual degradation as communication costs increase with problem irregularity.**

***Index Terms*—Sparse Matrix-Vector Multiplication, MPI, Distributed Memory, 1D Partitioning, Collective Communication, Strong Scaling, Weak Scaling**

I. Introduction

Problem. Sparse Matrix-Vector Multiplication (SpMV), computing $y = Ax$ where A is a sparse matrix and x is a dense vector, is a critical kernel in iterative solvers for linear systems [4] and eigenvalue problems, with applications in scientific and engineering simulations. Despite its computational simplicity ($O(nnz)$ work for nnz nonzeros), SpMV performance on distributed-memory systems is severely limited by irregular memory access patterns, load imbalance, and communication overhead [2], [6].

Objectives. This work aims to: (i) implement a distributed-memory SpMV using MPI with 1D cyclic partitioning and CSR storage format; (ii) optimize communication using `MPI_Alltoallv` collective operations with pre-computed patterns; (iii) evaluate strong scaling performance on real sparse matrices from SuiteSparse across 1–256 processes; (iv) assess weak scaling behavior on synthetic matrices; (v) identify fundamental scalability limits of 1D partitioning through detailed computation-communication breakdown analysis.

II. State of the Art

Parallel SpMV distributes both matrix A and vectors x, y across processes. In 1D row-based partitioning, each process owns a subset of matrix rows and corresponding vector entries. Cyclic (modulo) partitioning assigns row i to process $p = i \bmod P$, providing better load balance for irregular matrices than block partitioning but increasing communication [2]. For 1D partitioning, SpMV requires gathering remote vector elements where off-diagonal nonzeros a_{ij} with j owned by different processes require fetching x_j remotely. Communication strategies include point-to-point (`MPI_Isend/Irecv`, flexible but $O(p)$) messages and collective operations (`MPI_Alltoallv`, single call with potential all-to-all overhead). Boman et al. [1] demonstrated that 2D partitioning reduces message complexity from $O(p)$ to $O(\sqrt{p})$, enabling scaling to 16K+ cores on scale-free graphs. Our work provides detailed scalability analysis of pure 1D MPI implementation with cyclic partitioning, identifying the threshold where communication dominates and quantifying computation-communication breakdown.

III. Methodology

A. Algorithm and Data Structures

1) CSR Format: We use Compressed Sparse Row (CSR) format for local matrix storage:

- `val[]`: nonzero values (size: nnz_local)
- `col[]`: column indices (size: nnz_local)
- `rowptr[]`: row start offsets (size: $rows_local + 1$)

2) 1D Cyclic Partitioning: Row ownership is determined by: owner$(i) = i \bmod P$

Each process p owns rows $\{p, p + P, p + 2P, \ldots\}$ and computes corresponding entries of the result vector y. Local row indexing is computed as: local_row$(i) = i/P$ (integer division).

3) Distributed SpMV Algorithm:

B. Implementation Optimizations

1) Pre-computed Communication Patterns: Rather than determining communication needs at each SpMV call, we perform a one-time *setup phase* that: (1) identifies all remote column indices needed locally; (2) computes send/receive counts and displacements for `MPI_Alltoallv`; (3) builds mapping arrays to avoid runtime owner lookups; (4) pre-allocates communication buffers. This amortizes the cost of pattern analysis and enables efficient indexed access during computation.

Algorithm 1 Distributed SpMV with 1D Cyclic Partitioning

```
Setup Phase (once):
for each local column index j in A do
    owner ← j mod P
    if owner ≠ rank then
        Add j to needed[owner]
    end if
end for
Build send/recv counts and displacements
Create index mapping for remote elements

SpMV Phase (each iteration):
Pack send buffer with local x elements
MPI_Alltoallv(send_buf → recv_buf)
for each local row i do
    sum ← 0
    for k ← rowptr[i] to rowptr[i + 1] − 1 do
        if col[k] is local then
            sum ← sum + val[k] · x_local[local_idx[k]]
        else
            sum ← sum + val[k] · recv_buf[remote_idx[k]]
        end if
    end for
    y[i] ← sum
end for
```

2) Local vs. Remote Access Optimization: For each nonzero a_{ij}, we pre-compute `x_is_remote[k]` (boolean flag) and `x_idx[k]` (index into either x_{local} or $recv_buf$), avoiding repeated modulo operations and hash table lookups in the hot loop.

3) Communication via MPI_Alltoallv: We use `MPI_Alltoallv` instead of point-to-point messages because: (i) single collective call reduces software overhead; (ii) MPI implementation can optimize based on network topology; (iii) easier integration with timing instrumentation. However, this may send/receive empty messages to/from some processes, potentially increasing latency.

IV. Experimental Setup

A. Platform

- **CPU**: 4× Intel Xeon Gold 5220 (18 cores/socket, 72 cores total, 2.20 GHz base, 125W TDP)
- **Cache:** L1D: 32 KiB/core; L2: 1 MiB/core; L3: 24.75 MiB/socket
- **Memory:** 768 GB DDR4 RAM
- **Compiler:** GCC with `-O3`
- **MPI:** MPICH 3.2.1
- **Process Placement:** 1 MPI rank per core

B. Test Matrices

Strong Scaling: Real matrices from SuiteSparse Matrix Collection with high level of sparsity and diagonal form: 10K (bcsstk17), 40K (jnlbrng1), 108K (x104), 217K (pwtk), 377K (ML_Laplace), 504K (af_shell9), 1219K (CurlCurl_3), 2380K (CurlCurl_4). However in this paper we will focus on four of them: 40K, 504K, 1219K, 2380K

Weak Scaling: Synthetically generated sparse matrices with approximately constant row density, scaling from 10K to 2560K rows as process count increases from 1 to 256.

C. Metrics

Each configuration executed 50 SpMV iterations with detailed timing: **Total time** (end-to-end SpMV per iteration), **Communication time** (`MPI_Alltoallv` latency), **Computation time** (local CSR matrix-vector product), **Speedup** ($S_p = T_1/T_p$), and **Efficiency** ($E_p = S_p/p \times 100\%$). Timing via `MPI_Wtime()` with one warmup iteration.

V. Results and Analysis

A. Strong Scaling Performance

Table I summarizes strong scaling results.

TABLE I
Strong Scaling Results (Time in ms)

Matrix	Procs	Time	Speedup	Efficiency (%)
40K	1	140.50	1.00×	100.0
	2	195.19	0.72×	36.0
	4	99.22	1.42×	35.4
504K	1	15,029	1.00×	100.0
	16	3,059	4.91×	30.7
	32	2,178	6.90×	21.6
	64	**1,567**	**9.59×**	**15.0**
1219K	1	10,322	1.00×	100.0
	32	1,993	5.18×	16.2
	64	1,281	8.06×	12.6
	128	229,677	0.04×	0.04
2380K	1	26,615	1.00×	100.0
	32	4,614	5.77×	18.0
	64	3,403	7.82×	12.2
	128	38,831	0.69×	0.54
	256	618,913	0.04×	0.02

Key observations: (1) Good scaling up to 64 processes with 9.59× speedup on 504K matrix; (2) Catastrophic degradation at 128–256 processes—execution time increases 179× (1219K) and 483× (2380K) vs. 64-process runs; (3) Efficiency drops significantly with process count even for well-scaling matrices.

Figure 1 shows speedup curves, illustrating the scalability limit.

B. Computation vs. Communication Breakdown

Figures 2 and 3 show time decomposition. Table II quantifies the breakdown.

Computation time decreases as expected with added processes, but communication time initially increases gradually (up to 64 processes) then explodes exponentially. At the scalability limit, computation becomes negligible ($< 1\%$ of runtime). This behavior is consistent with 1D partitioning's $O(p)$ message complexity—communication volume does not

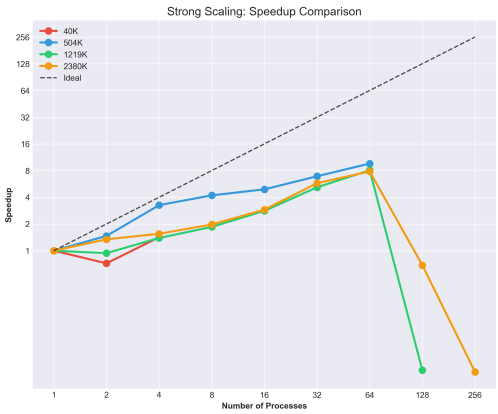

Fig. 1. Strong scaling speedup comparison. Dashed line shows ideal linear speedup. Note severe degradation for 1219K and 2380K matrices at high process counts.

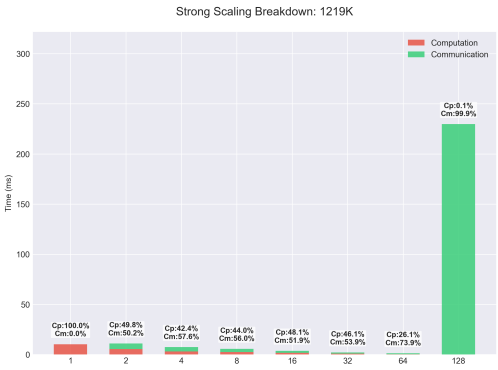

Fig. 2. Computation/Communication breakdown for 1219K matrix. At 128 processes, computation drops to 0.1% of total time while communication dominates at 99.9%.

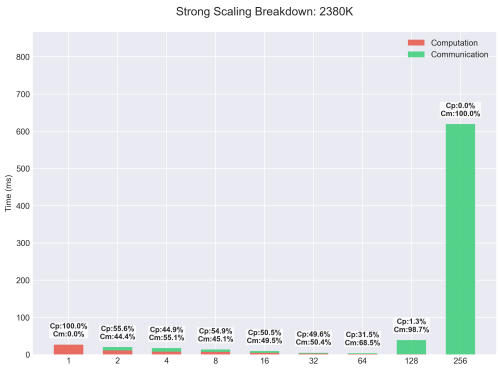

Fig. 3. Computation/Communication breakdown for 2380K matrix. Similar pattern observed, with near-complete communication domination at 256 processes.

TABLE II
COMMUNICATION OVERHEAD ANALYSIS

Matrix	Procs	Comp (ms)	Comm (ms)	Comm %
1219K	1	10,318	3.0	0.0%
	32	920	1,073	53.9%
	64	334	947	73.9%
	128	**152**	**229,525**	**99.9%**
2380K	1	26,611	3.5	0.0%
	32	2,288	2,326	50.4%
	64	1,072	2,331	68.5%
	128	523	38,307	98.7%
	256	**212**	**618,700**	**100%**

decrease proportionally with problem size per process, leading to communication-bound execution.

C. Efficiency Heatmap

Figure 4 visualizes parallel efficiency across all configurations, showing: (1) near-perfect efficiency for small matrices at low process counts; (2) gradual degradation as scale increases; (3) catastrophic failure for large matrices at high process counts; (4) a diagonal "efficiency cliff" beyond which 1D partitioning becomes ineffective.

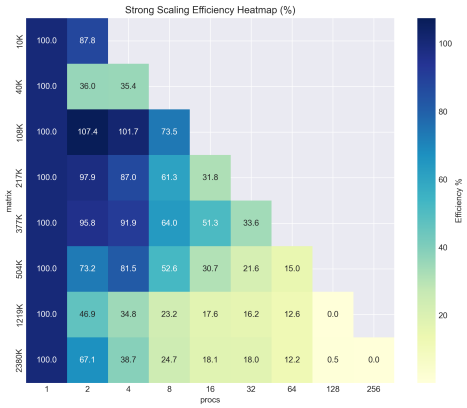

Fig. 4. Strong scaling efficiency heatmap (%). Dark blue indicates good efficiency ($> 80\%$), while lighter colors show degradation.

D. Weak Scaling Analysis

Weak scaling results (Figures 5 and 6) show: near-ideal scaling up to 32 processes (runtime factor < 1.0 due to cache effects), gradual degradation from 32 to 128 processes ($1.4\times$–$2.3\times$ overhead), and severe degradation at 256 processes ($4.5\times$ overhead). Communication fraction increases from 69.8% (1P) to 97.8% (256P), confirming that overhead grows with process count even when problem size scales proportionally, due to increased network diameter, higher contention, and per-message software overheads.

E. Discussion

1) Why 1D Partitioning Fails at Scale: 1D row-based partitioning suffers from $O(p)$ message complexity. For irregular

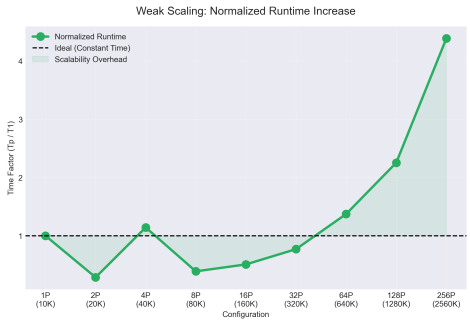

Fig. 5. Weak scaling: normalized runtime increase. Ideal weak scaling maintains constant time (dashed line at 1.0).

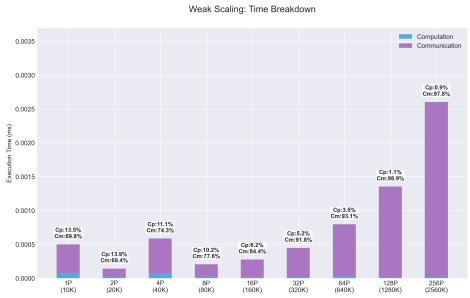

Fig. 6. Weak scaling: computation vs. communication time breakdown. Note increasing communication fraction with process count.

matrices with nnz nonzeros across p processes: each process owns $\approx nnz/p$ nonzeros, many are off-diagonal, and in the worst case each process communicates with all $p-1$ others, creating $O(p^2)$ total messages. Even with `MPI_Alltoallv`, the all-to-all pattern incurs latency proportional to p.

2) When Does 1D Work Well?: Our results show 1D partitioning is effective when: (i) small to moderate process counts ($p \leq$ 32–64); (ii) sufficient local work (computation exceeds communication); (iii) structured matrices (regular patterns reduce off-diagonal communications). The 504K matrix achieved nearly 10× speedup on 64 processes, demonstrating viability at this scale.

3) Comparison with 2D Partitioning: Theoretical analysis [1], [3] predicts 2D partitioning reduces message complexity to $O(\sqrt{p})$ by arranging processes in a $\sqrt{p} \times \sqrt{p}$ grid and limiting communication to row/column neighbors. For $p = 256$, this reduces maximum messages from 255 (1D) to $2\sqrt{256} = 32$ (2D), a nearly 8× reduction. Our optimizations (pre-computed patterns, local/remote index arrays, single `MPI_Alltoallv`) successfully reduced per-iteration overhead but cannot overcome fundamental $O(p)$ message complexity—the performance cliff at 128–256 processes is a *scalability limit*, not an implementation inefficiency.

4) Impact of Matrix Structure: Matrix structure significantly affects scaling: finite element matrices (1219K, 2380K CurlCurl) with localized dependencies scale better than expected up to 64 processes, though larger problem size increases communication diameter. For truly scale-free matrices (power-law graphs), we expect worse 1D scaling due to high-degree hub vertices creating heavy inter-process communication.

5) Optimization Effectiveness: Our optimizations successfully reduced per-iteration overhead: Pre-computed patterns eliminate runtime hash lookups, Local/remote index arrays avoid modulo operations in hot loop, Single `MPI_Alltoallv` reduces software overhead vs. many point-to-point messages.
However, these optimizations cannot overcome the fundamental $O(p)$ message complexity. The performance cliff at 128–256 processes is a *scalability limit*, not an *implementation inefficiency*. Similar patterns arise in SpGEMM [5], where 2D methods prove essential.

VI. Conclusions

We presented a distributed-memory SpMV implementation with 1D cyclic partitioning and optimized collective communication, evaluating strong and weak scaling on sparse matrices up to 2.38M rows using up to 256 MPI processes.

Key contributions: (1) Demonstrated that 1D cyclic partitioning with `MPI_Alltoallv` achieves good strong scaling up to 64 processes (9.59× speedup on 504K matrix); (2) Identified catastrophic scalability failure beyond 64 processes, with communication overhead reaching $> 99\%$ of runtime; (3) Quantified computation-communication breakdown, showing optimization cannot overcome fundamental $O(p)$ message complexity; (4) Confirmed 1D partitioning is unsuitable for large-scale distributed computing on irregular sparse matrices.

Implications: For production SpMV, our results strongly motivate adoption of 2D partitioning methods [1] that reduce message complexity to $O(\sqrt{p})$. The performance cliff at 128–256 processes represents a hard scalability limit for 1D approaches.

Future work: Implement 2D Cartesian partitioning; evaluate hybrid MPI+OpenMP to overlap communication with computation; test on scale-free graphs.

VII. Reproducibility & Artifact Availability

Git.
https://github.com/11ila/PARCO-Computing-2026-244528/blob/D2 (branch: D2)

How to Run.

```
# Compile
mpicxx -O3 -std=c++11 -o code src/code.cpp
# Execute benchmarks
qsub scripts/strong_scaling.pbs
qsub scripts/weak_scaling.pbs
# Generate plots
python3 scripts/graph_generator.py
```

For cluster execution: modify PBS scripts. Full compilation and submission details in repository README.

Inputs: Strong scaling uses eight SuiteSparse matrices (10K–2380K rows). Weak scaling uses nine synthetic matrices (10K–2560K rows). All matrix details under `scripts/matrix/`.

Outputs: CSV files (`results/strong_scaling_results.csv`, `weak_scaling_results.csv`) with timing breakdowns, per-run PBS logs, and performance plots (speedup, efficiency, computation/communication) in `plots/`.

REFERENCES

[1] E. G. Boman, K. D. Devine, and S. Rajamanickam, "Scalable matrix computations on large scale-free graphs using 2D graph partitioning," in *Proc. SC '13*, 2013, pp. 50:1–50:12.

[2] G. Schubert, G. Hager, H. Fehske, and G. Wellein, "Parallel sparse matrix-vector multiplication as a test case for hybrid MPI+OpenMP programming," *arXiv:1101.0091*, 2011.

[3] A. Yoo, A. H. Baker, R. Pearce, and V. E. Henson, "A scalable eigensolver for large scale-free graphs using 2D graph partitioning," in *Proc. SC '11*, 2011, pp. 63:1–63:11.

[4] Y. Saad, *Iterative Methods for Sparse Linear Systems*, 2nd ed. SIAM, 2003, ch. 3, 6.

[5] A. Buluç and J. R. Gilbert, "Parallel sparse matrix-matrix multiplication and indexing: Implementation and experiments," *SIAM J. Sci. Comput.*, vol. 34, no. 4, pp. C170–C191, 2012.

[6] U. V. Çatalyurek and C. Aykanat, "Hypergraph-partitioning-based decomposition for parallel sparse-matrix vector multiplication," *IEEE Trans. Parallel Distrib. Syst.*, vol. 10, no. 7, pp. 673–693, 1999.